Przetwarzanie strumieni danych

Spark Streaming

\+ Apache Kafka

Do tej pory zajmowaliśmy się przetwarzaniem dużych zbiorów danych, które już znajdują się w klastrze Hadoop.

Pytanie: ⇨ jak dane trafiają na HDFS?

Przetwarzanie strumieni danych ⇨ to proces publikowania danych z jakiegoś źródła np.:
- logów sieciowych
- danych z czujników IoT

Często zależy nam również na przetwarzaniu takich danych w czasie rzeczywistym.

tzn. na przetwarzaniu nowych danych w miarę ich napływania bez konieczności ręcznego ładowania tych danych (w dużych porcjach) w określonych interwałach czasowych.

Pytanie: ⇨ jak publikować dane ze źródeł danych w klastrze w czasie rzeczywistym, aby można je było następnie przetworzyć.

odp. np. przez wykorzystane brokera

Kafka ⇨ to jest system przesyłania wiadomości do publikowania/subskrybowania.

zadaniem klastra serwerów Kafka jest przechwycenie i przechowywanie wszystkich wiadomości przychodzących od wydawców (np. od wiązki serwerów internetowych lub wiązki czujników) i publikowanie tych danych każdemu, kto chce je konsumować.

Jest wiele zastosowań tego typu, np. monitorowanie danych dotyczące zachowań klientów pochodzących z logów na serwerach internetowych, można przekształcić te dzienniki w struktury baz danych.

Przetwarzanie strumieni danych powoduje wyzwania:
- jak przenieść dane ze źródeł danych do klastra w sposób skalowalny i niezawodny?
- jak przetworzyć te dane by uzyskać wartościową informację?

APACHE kafka
A distributed streaming platform

Zbieranie Metryk

Jak to w praktyce działa?

https://softwareskill.pl/apache-kafka-wprowadzenie



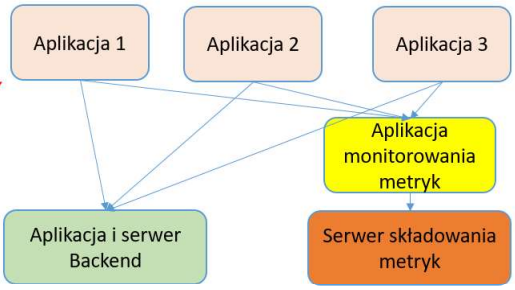
załóżmy, że system składa się z wielu aplikacji front-end.

oraz... ...aplikacji służącej do zbierania metryk z tych aplikacji

usługi frontend → udostępniają dane klientowi

... oraz same produkują dane:

np.
- czasy wykonania zapytań
- co klient robił na stronie
- ...w tym sekwencje zdarzeń, ich czasy, ilość wykonanych operacji, zasad naruszania bezpieczeństwa ...itd



Metryki Są bardzo istotnym elementem zbierania informacji do dalszej analizy dla działów IT i marketingu i z tego powodu gromadzi się je w bazach danych o różnej topologii i relacjach.

z rozwojem systemu, aplikacji produkujących metryki, jak i samych metryk pojawia się na tyle dużo, że należy zaplanować cały system ich przepływu i przetwarzania

... a bezpośrednia komunikacja pomiędzy mikrousługami jest na tyle skomplikowana, że należy zaplanować system przesyłania i transmisji komunikatów

Model Publish-Subscribe

Model Publish-Subscribe → jak w praktyce działa?

https://softwareskill.pl/apache-kafka-wprowadzenie

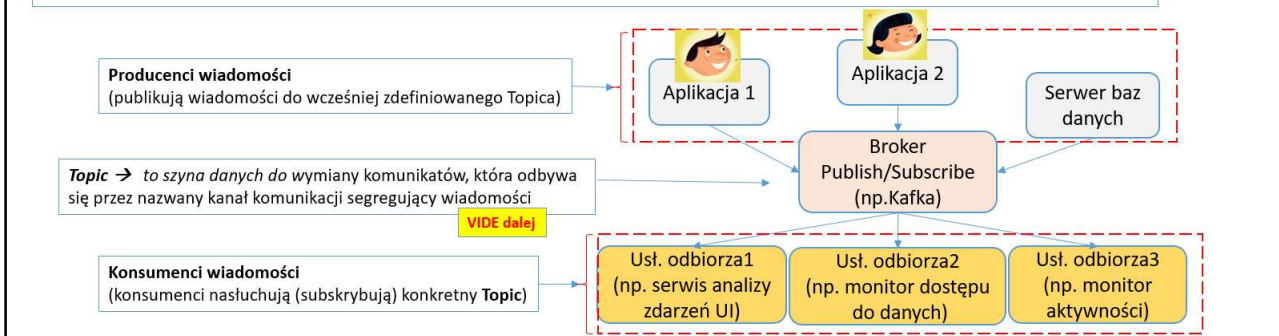
W miejsce bezpośrednich integracji mikrousług ze sobą, które stają się osobnym wyzwaniem - można wprowadzić pośrednika tzw. **brokera**.

Nowe komunikaty związane z monitorowaniem działania mikrousług przechodzą przez komunikację z pośrednikiem więc posiadają jeden punkt integracji.

Wszyscy, którzy chcą odczytywać komunikaty, subskrybują komunikaty przez pośrednika.

Twórca publikuje treści ⇨ zainteresowani odbiorcy (subskrybenci) odbierają treści, po ich opublikowaniu (lub w swoim wolnym czasie).

Przykładowo - serwis społecznościowy pełni tu rolę pośrednika, wyręczając twórcę treści od „ręcznego" powiadomienia tysięcy zainteresowanych.



Jeśli konsument zasubskrybuje się w konkretnym topicu – będzie dostawał informację, kiedy jakaś wiadomość się na nim pojawi.

Powoduje to separację: ⇨ Producent nie musi wiedzieć o istnieniu konsumenta oraz kim jest

Jest to tzw. system kolejkowy komunikatów

https://softwareskill.pl/apache-kafka-wprowadzenie

Przykłady systemów kolejkowych:

Zmiany notowań w aplikacji giełdowej ⇨ w której odbywa się subskrypcja na **topicu** informującym o zmianach wartości akcji, walut, obligacji… (bez konieczności odpytywania serwer co kilka ms o aktualny kurs.)

Inernet Of Things (IoT) ⇨
- monitoring GPS pojazdów – co jakiś czas urządzenie wysyła na kolejkę informacje o aktualnej pozycji GPS.
- np. sensory temperatury, opadów mogą co jakiś czas wrzucać na kolejkę informacje o aktualnym stanie pomiarów, a system subskrybujący (konsumer) może na podstawie tych danych rysować wykresy temperatury

Wyzwania dla systemów kolejkowych:
- szybkość
- bezpieczeństwo
- bezawaryjność
- zapewnianie prostego API
- możliwie prosta konfiguracja

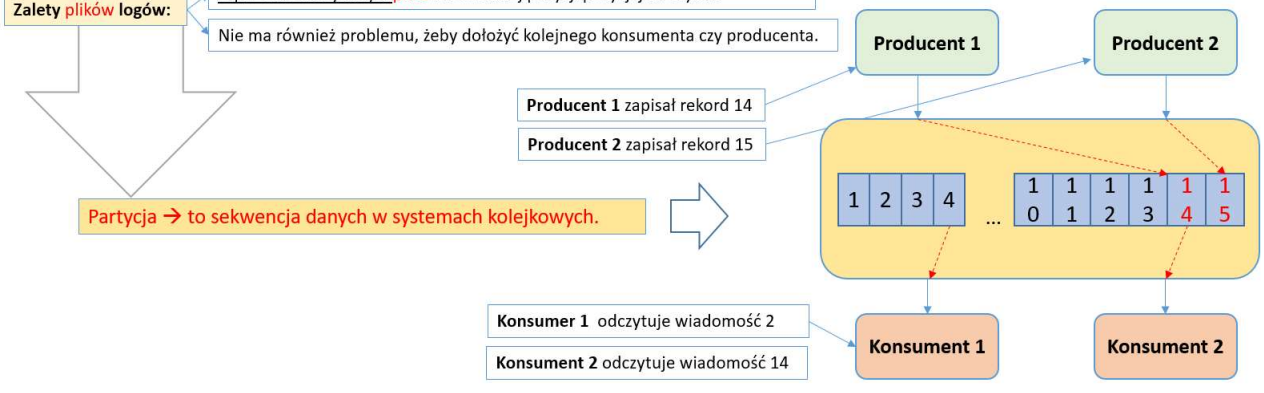
https://softwareskill.pl/apache-kafka-wprowadzenie

Czym jest Partycja?

Ponieważ **Konsument** odczytuje ostatnie dane, których jeszcze nie przeczytał ➔ wyzwaniem w systemach kolejkowych jest możliwość dokładania danych na końcu, bez ich aktualizowania

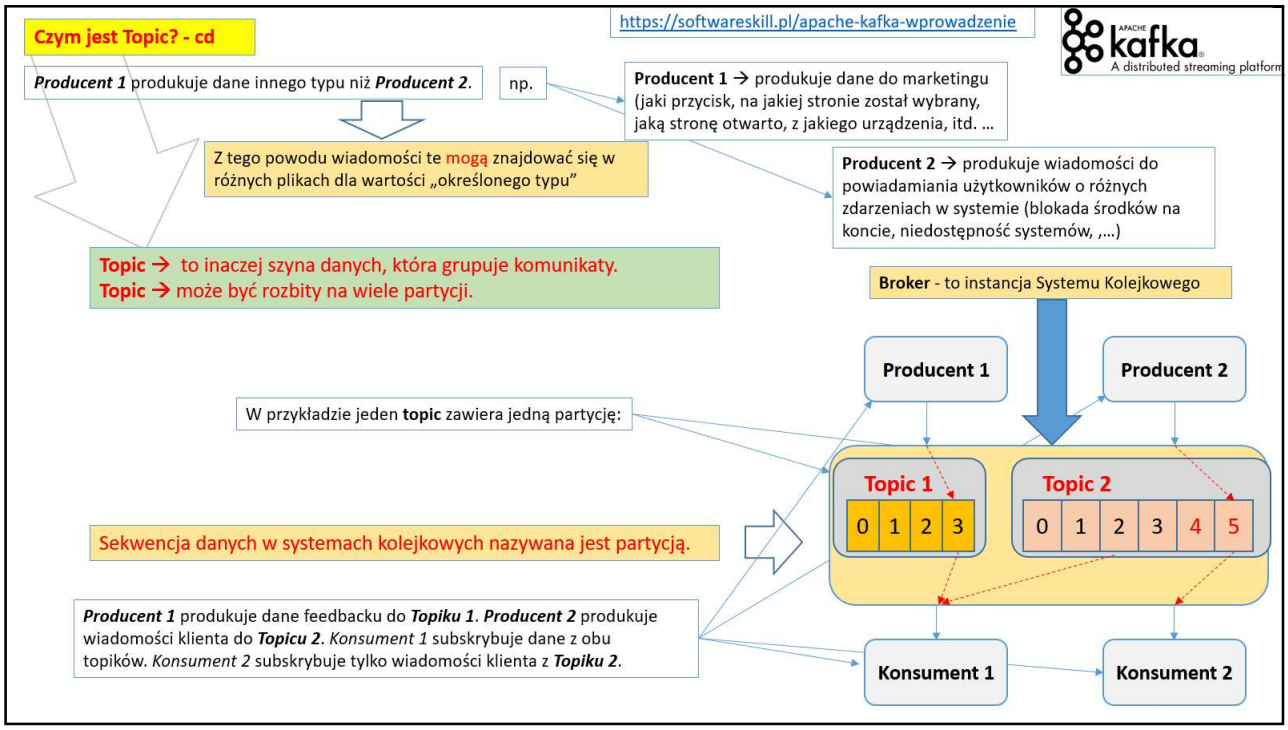
Filozofia zapisu oparta jest na… ⇨ **Logach aplikacji**. (plikach logów) ⇨ które aplikacja produkuje i odkłada w plikach. ⇨ dzięki temu można się „za-subskrybować" do konkretnego loga i patrzeć co nowego się w takim logu pojawiło.

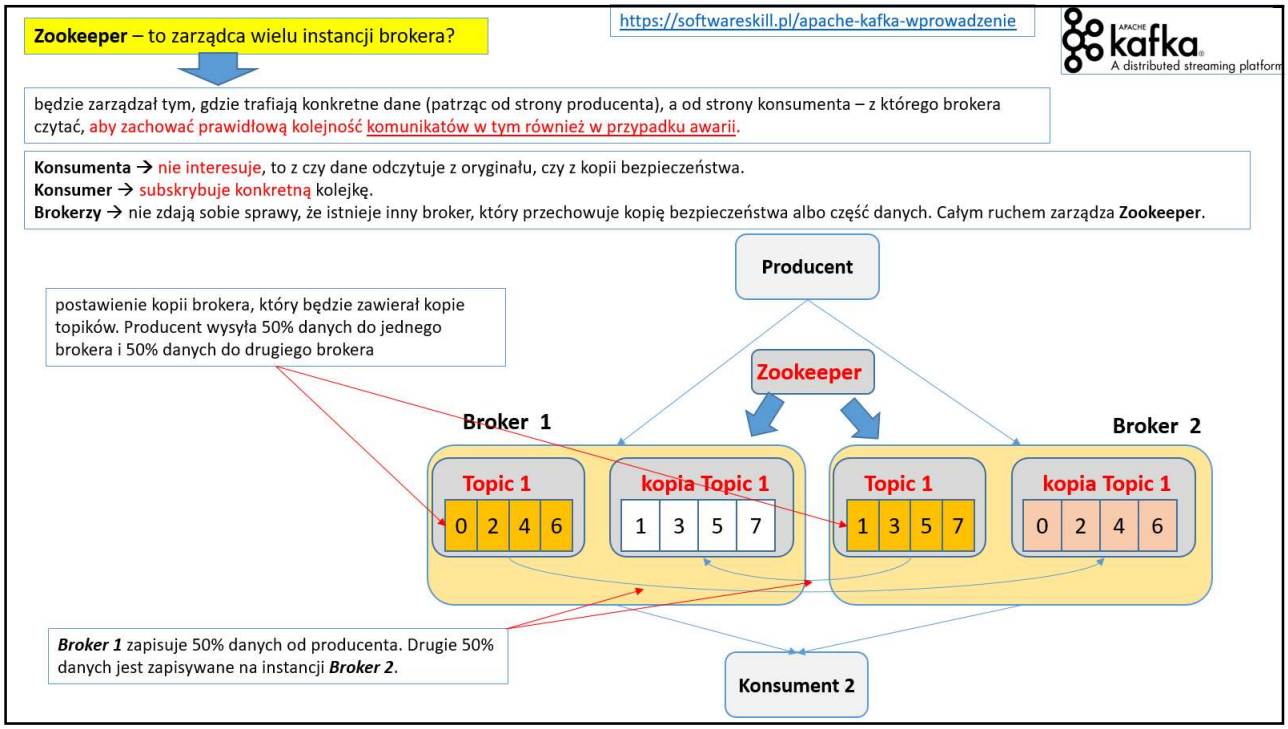
Zalety plików logów:
- zapis oraz odczyt do/z pliku na ostatniej pozycji pozycji jest szybki.
- Nie ma również problemu, żeby dołożyć kolejnego konsumenta czy producenta.



Partycja ➔ to sekwencja danych w systemach kolejkowych.

Pytanie: skąd konsumenci mają wiedzieć, jakie ostatnie dane zostały odczytane przy awarii? ⇨ Odpowiedź – VIDE dalej


Czym jest Topic? - cd
https://softwareskill.pl/apache-kafka-wprowadzenie
kafka
A distributed streaming platform
Producent 1 produkuje dane innego typu niż Producent 2.
np.
Producent 1 → produkuje dane do marketingu (jaki przycisk, na jakiej stronie został wybrany, jaką stronę otwarto, z jakiego urządzenia, itd. ...
Z tego powodu wiadomości te mogą znajdować się w różnych plikach dla wartości „określonego typu"
Producent 2 → produkuje wiadomości do powiadamiania użytkowników o różnych zdarzeniach w systemie (blokada środków na koncie, niedostępność systemów, ,...)
Topic → to inaczej szyna danych, która grupuje komunikaty.
Topic → może być rozbity na wiele partycji.
Broker - to instancja Systemu Kolejkowego
Producent 1
Producent 2
W przykładzie jeden topic zawiera jedną partycję:
Topic 1
0 1 2 3
Topic 2
0 1 2 3 4 5
Sekwencja danych w systemach kolejkowych nazywana jest partycją.
Producent 1 produkuje dane feedbacku do Topiku 1. Producent 2 produkuje wiadomości klienta do Topicu 2. Konsument 1 subskrybuje dane z obu topików. Konsument 2 subskrybuje tylko wiadomości klienta z Topiku 2.
Konsument 1
Konsument 2


Zookeeper – to zarządca wielu instancji brokera?
https://softwareskill.pl/apache-kafka-wprowadzenie
kafka
A distributed streaming platform
będzie zarządzał tym, gdzie trafiają konkretne dane (patrząc od strony producenta), a od strony konsumenta – z którego brokera czytać, aby zachować prawidłową kolejność komunikatów w tym również w przypadku awarii.
Konsumenta → nie interesuje, to z czy dane odczytuje z oryginału, czy z kopii bezpieczeństwa.
Konsumer → subskrybuje konkretną kolejkę.
Brokerzy → nie zdają sobie sprawy, że istnieje inny broker, który przechowuje kopię bezpieczeństwa albo część danych. Całym ruchem zarządza Zookeeper.
Producent
postawienie kopii brokera, który będzie zawierał kopie topików. Producent wysyła 50% danych do jednego brokera i 50% danych do drugiego brokera
Zookeeper
Broker 1
Broker 2
Topic 1
0 2 4 6
kopia Topic 1
1 3 5 7
Topic 1
1 3 5 7
kopia Topic 1
0 2 4 6
Broker 1 zapisuje 50% danych od producenta. Drugie 50% danych jest zapisywane na instancji Broker 2.
Konsument 2

offset

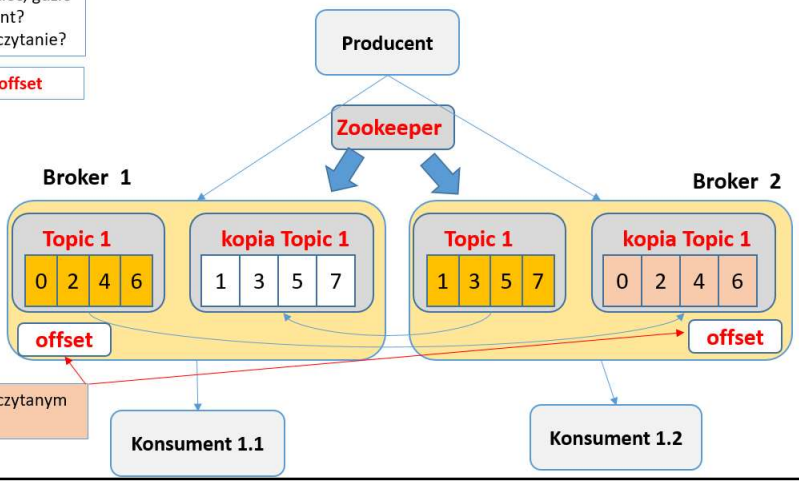
https://softwareskill.pl/apache-kafka-wprowadzenie

przechowuje informacje o ostatnio odczytanym rekordzie.

Przykładowo: mamy dwóch konsumentów tego samego topika – Konsument 1.1 i Konsument 1.2. Posiadają oni ten sam kod, a ruch jest rozdzielany po równo pomiędzy instancje konsumentów. Podczas bezawaryjnej pracy systemu, Konsument 1.1 odczytuje 50% danych, a Konsument 1.2 odczytuje pozostałe 50% danych. W przypadku awarii Konsumenta 1.1 – drugi konsument przejmuje obsługę wszystkich wiadomości.

Pytanie:
- Skąd drugi konsument ma wiedzieć, gdzie skończył czytać pierwszy konsument?
- od jakiej wiadomości ma zacząć czytanie?

Odp. Informacja ta znajduje się w **offset**



offset → przechowuje informacje o ostatnio odczytanym rekordzie.

czym jest ⇨ Apache Kafka ?

Konsumenci zasadniczo subskrybują jeden lub więcej tematów i dzięki serwerom Kafka będą otrzymywać dane w miarę ich publikacji.

Zaletą Apache Kafka jest to, że przechowuje (buforuje) dane, dzięki czemu konsumenci mogą nadrobić zaległości w ich subskrypcji (pobieraniu) od miejsca, w którym ostatnio przerwali, więc utrzymuje punkt, w którym każdy konsument przerwał pobieranie danych i pozwoli im odebrać te dane kiedy będzie to możliwe.

Czym jest **Apache Kafka?**

- broker wiadomości dostępny jako otwarte oprogramowanie
- Projekt jest napisany w Scali, a rozwija go Apache Software Foundation
- umożliwia obsługę danych czasu rzeczywistego pochodzących z wielu węzłów. Zapewnia ujednoliconą obsługę strumieni wejściowych przy dużej przepustowość i redukcji czasów opóźnień. Projekt powstał pod silnym wpływem dzienników transakcji.
- Kafka początkowo zaprojektowano w LinkedIn, a następnie udostępniono jako open source na początku 2011 roku.

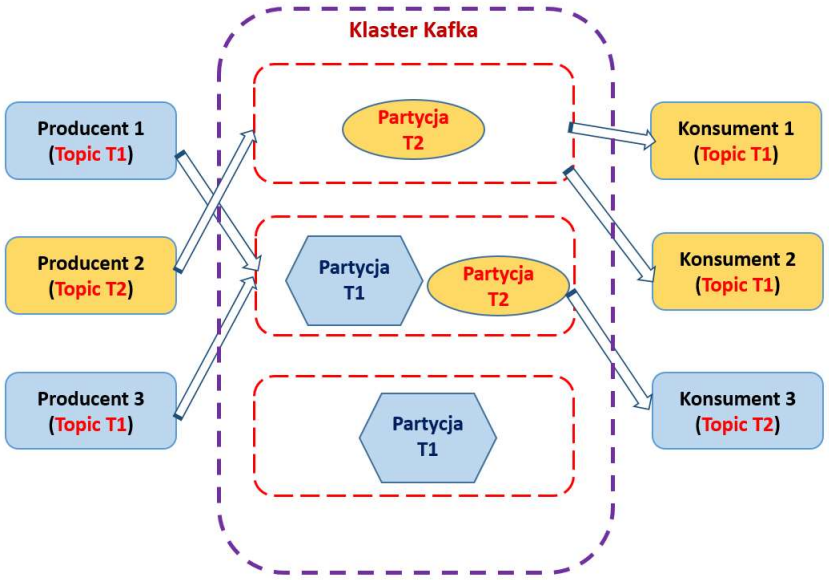
Apache Kafka jest rozproszoną platformą do przesyłania milionów wiadomości.

w Apache Kafka posługujemy się najczęściej pojęciem logu, chociaż czasem zamiennie używa się też słowa wiadomość lub event.

Każdy nowy log(wiadomość/event) jest dopisywany na końcu pliku.

Pozwoliła firmie LinkedIn na przetworzenie ok. 7 milionów komunikatów na sekundę!

Partycje są replikowanie, czyli ich kopie mogą znajdować się na wielu serwerach. Dla każdej partycji istnieje jeden serwer, który pełni funkcję tzw. **lidera** – obsługuje on wszystkie operacje zapisu i odczytu na danej partycji. Jeśli partycji jest więcej (np. poziom replikacji ustawiony na 3) – będziemy mieli jednego lidera i 2 brokerów, którzy tylko kopiują dane na 2 pozostałe partycje, w celu zwiększenia bezpieczeństwa klastra.



klaster Kafki, który składa się z:

trzech brokerów: B1, B2, B3
dwa topiki (T1 i T2)
dwie partycje (po jednej na topik)
trzech producentów wiadomości
trzech konsumentów wiadomości

Broker B2 jest liderem dla topiku T1. Broker B1 jest liderem dla topiku T2. Partycja T2 na brokerze B2 jest kopią partycji z brokera B1. Partycja na brokerze B3 jest kopią partycji T1 z brokera B2. W przypadku awarii borkera B2 rolę lidera dla topika T1 przejmie B3. Ponadto zostaną utworzone nowe kopie partycji T1 i T2 na działających instancjach brokera.

Ekosystem Apache Kafka

Niezbędnym elementem, każdego klastra Apache Kafka jest Zookeeper. Minimum jeden, a zazwyczaj trzy nody, żeby zapewnić wysoką dostępność i tolerancje na awarie.

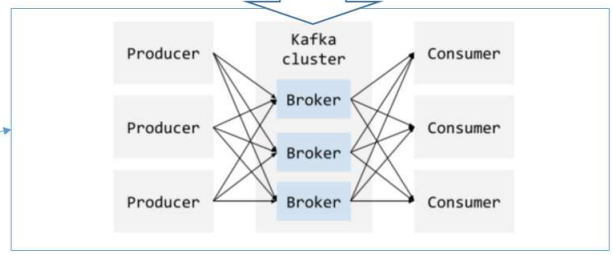
Apache Kafka → trzyma w **Zookeeper** metadane dotyczące:

- **topic**, (grup wiadomości)
- **borker**,
- **consumer**.

UWAGA !!!

Zookeeper nie ma wysyłanych danych tylko metadane klastra.

Relację pomiędzy **producer**, **broker** i **consumer**:



Topic → to grupa wiadomości.

Każdy **producer** może pisać do każdego brokera oraz każdy **consumer** może konsumować wiadomości z każdego **brokera**.

Jeden **producer** może pisać do wielu grup wiadomości, tak samo **consumer** może czytać dane z wielu grup wiadomości.

Grupa wiadomości (tzw. topic) jest podzielona na **partycje**, a każda **partycja** zawiera jakąś cześć wiadomości, które zostały wysłane.

Po co są partycje ? ⇨ ...odp. ⇨ Do zwiększenia przepustowości transmisji.

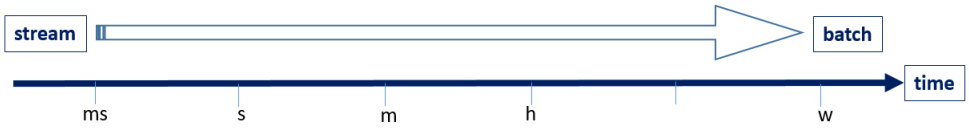
Załóżmy, że producer wysyła dane do grupy wiadomości, która ma 3 partycje a każda z nim jest na innym borkerze. Tym samym wysyłamy dane do trzech różnych serwerów, przez trzy różne karty sieciowe, procesory i na różne dyski. W teorii mamy trzykrotnie większą przepustowość.

Przetwarzanie strumieni danych

Spark Streaming

Spark - Przetwarzanie danych

Spark ⇨ łączy dwa podejścia:
- Przetwarzanie strumieni
- Przetwarzanie batch



Przetwarzanie strumieni danych co do filozofii jest bardzo podobna do przetwarzania batch

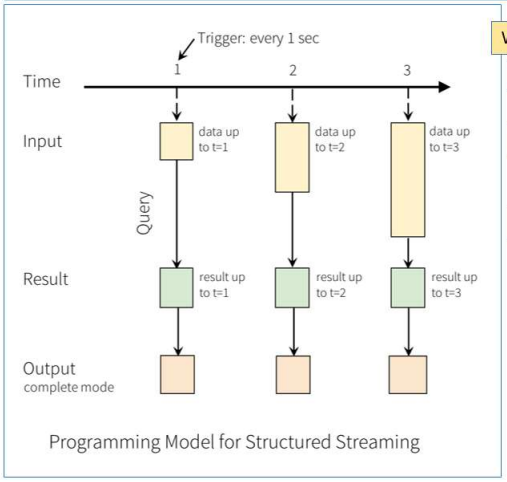
Różnice w ogólnym zarysie polegają jak szybko można pozyskać i przetworzyć dane tak, by otrzymać rezultaty w postaci wyniku.

Dane te mogą być pochodzić z wielu źródeł:



Streaming→ Spark 3.1.2

Zapytanie o dane wejściowe wygeneruje „Tabelę Wyników". Z każdym interwałem wyzwalacza (powiedzmy co 1 sekundę) nowe wiersze są dołączane do „Tabeli Wejściowej", co ostatecznie aktualizuje „Tabelę Wyników". Za każdym razem, gdy tabela wyników zostanie zaktualizowana, chcielibyśmy zapisać zmienione wiersze wyników do zewnętrznego wyjścia przez serię małych zadań wsadowych.



Programming Model for Structured Streaming

Wyjście można zdefiniować w trybie:

Tryb kompletny	cała zaktualizowana tabela wyników zostanie zapisana w pamięci zewnętrznej. Od pamięci masowej zależy, jak obsłużyć zapis całej tabeli
Tryb dołączania	tylko nowe wiersze dołączone w tabeli wyników od ostatniego wyzwalacza zostaną zapisane w pamięci zewnętrznej. Ma to zastosowanie tylko do zapytań, w których nie oczekuje się zmiany istniejących wierszy w tabeli wyników.
Tryb aktualizacji	tylko wiersze, które zostały zaktualizowane w tabeli wyników od czasu ostatniego wyzwalacza, zostaną zapisane w magazynie zewnętrznym (dostępne od wersji Spark 2.1.1). Zauważ, że różni się on od trybu pełnego tym, że ten tryb wyświetla tylko wiersze, które uległy zmianie od ostatniego wyzwalacza. Jeśli zapytanie nie zawiera agregacji, będzie równoważne z trybem dołączania.

Inicjalizacja → StreamingContext (do Spark 3.1.2)

Najpierw musimy zaimportować potrzebne klasy i stworzyć sesję lokalną **SparkSession**, czyli punkt wejścia dla wszystkich funkcjonalności.

Scala:

```
import org.apache.spark.sql.functions._
import org.apache.spark.sql.SparkSession
val spark = SparkSession
.builder
.appName("StructuredNetworkWordCount")
.getOrCreate()
import spark.implicits._
```

Python:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode
from pyspark.sql.functions import split
spark = SparkSession \
 .builder \
 .appName("StructuredNetworkWordCount") \
 .getOrCreate()
```

Następnie utwórzmy ramkę DataFrame przesyłania strumieniowego, która reprezentuje dane tekstowe odebrane z serwera nasłuchującego na hoście lokalnym:9999, i przekształć ramkę DataFrame w celu obliczenia liczby słów.

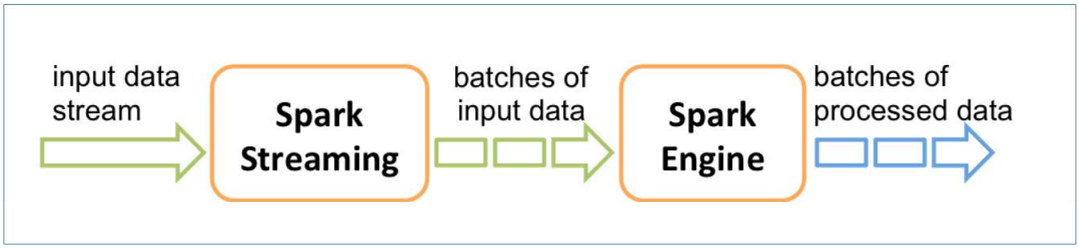
```
// Create DataFrame representing the stream of input lines from
connection to localhost:9999
val lines = spark.readStream
 .format("socket")
 .option("host", "localhost")
 .option("port", 9999)
 .load()
// Split the lines into words
val words = lines.as[String]
 .flatMap(_.split(" "))
// Generate running word count
val wordCounts = words
 .groupBy("value")
 .count()
```

```
# Create DataFrame representing the stream of input lines from
connection to localhost:9999
 lines = spark \
 .readStream \ .
("socket") \
 .option("host", "localhost") \
 .option("port", 9999) \
 .load()
# Split the lines into words
words = lines.select( explode( split(lines.value, " ") ).alias("word") )
# Generate running word count
wordCounts = words.groupBy("word").count()
```

Spark Streaming (od Spark 3.2) - jak działa?

Spark Streaming (od Spark 3.2) - jak to działa?

Spark Streaming odbiera strumienie danych wejściowych w czasie rzeczywistym i dzieli je na partie, które są następnie przetwarzane przez aparat Spark w celu wygenerowania końcowego strumienia wyników w partiach.



Spark Streaming zapewnia abstrakcję wysokiego poziomu o nazwie ***discretized stream*** lub ***DStream*** , która reprezentuje ciągły strumień danych. **DStreams** można tworzyć ze strumieni danych wejściowych ze źródeł takich jak Kafka i Kinesis lub przez zastosowanie operacji wysokiego poziomu na innych **DStreams**. Wewnętrznie **DStream** jest reprezentowany jako sekwencja RDD .

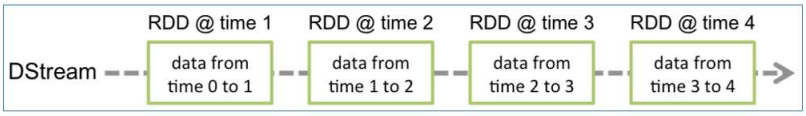
Dyskretne strumienie (DStreams)(od Spark 3.2)

DStream to podstawowa abstrakcja udostępniana przez Spark Streaming.

Reprezentuje:

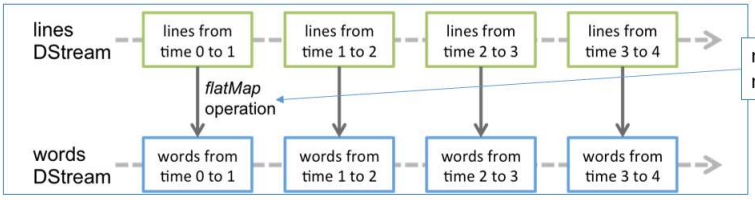
- ciągły strumień danych wejściowych odebrany ze źródła,
- przetworzony strumień danych wygenerowany przez przekształcenie strumienia wejściowego.

Wewnętrznie **DStream** jest reprezentowany przez ciągłą serię **RDD**, która jest abstrakcją Spark niezmiennego, rozproszonego zestawu danych.



Każdy **RDD** w **DStream** zawiera dane z określonego interwału.

Każda operacja zastosowana na **DStream** przekłada się na operacje na podstawowych **RDD**



np. konwertowanie strumienia wierszy na słowa z wykorzystaniem **flatMap**

Operacje **DStream** ukrywają większość tych szczegółów i zapewniają programiście interfejs API wyższego poziomu dla wygody.

Inicjalizacja → StreamingContext (DStreams)(od Spark 3.2)

Obiekt **StreamingContext** można utworzyć z obiektu **SparkConf** .

Obiekt **StreamingContext** → jest głównym punktem wejścia wszystkich funkcji Spark Streaming.

Scala:

```
import org.apache.spark._
import org.apache.spark.streaming._

val conf = new SparkConf().setAppName(appName).setMaster(master)
val ssc = new StreamingContext(conf, Seconds(1))
```

Python:

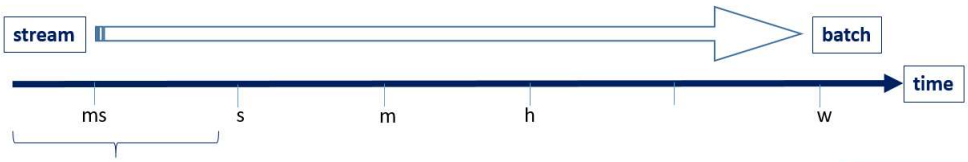
```
from pyspark import SparkContext
 from pyspark.streaming import StreamingContext
sc = SparkContext(master, appName)
ssc = StreamingContext(sc, 1)
```

Uwaga:

- Po uruchomieniu kontekstu nie można konfigurować ani dodawać do niego nowych obliczeń przesyłania strumieniowego.
- Po zatrzymaniu kontekstu nie można go ponownie uruchomić.
- Tylko jeden **StreamingContext** może być aktywny w **JVM** w tym samym czasie.
- **stop()** w **StreamingContext** również zatrzymuje **SparkContext**. Aby zatrzymać tylko **StreamingContext**, ustaw opcjonalny parametr stop() na false w wywołaniu dla SparkContext.

Spark - Przetwarzanie danych

To pierwsze narzędzie łączące przetwarzanie strumieniowe i batch w podobny sposób



Obecnie Spark nie pozwala na przetwarzanie danych w czasie poniżej milisekund, chociaż istnieje eksperymentalne ciągłe przetwarzanie danych jednak na obecną chwilę nie jest ono jeszcze zalecane.

```
(
  df.select("X", "Y")
    .where("Y is not null")
    .groupBy("X").count()
)
```

- wybieramy podzbiór kolumn
- służy selekcji i filtrowaniu
- służy do grupowania danych

Klasyczny df i jego transformacje

Klasyczne przetwarzanie **batch** oferuje bardzo podobny interfejs co w przypadku przetwarzania strumieni danych

Te same (lub bardzo podobne) komendy biblioteki SQL można wykorzystać na strumieniach lub na statycznych danych

Wniosek:

- można bardzo szybko przeprowadzić logikę przetwarzania na strumieniach znając przetwarzanie batch
- wystarczy zastąpić początek i koniec definicji aplikacji by generalnie wszystko działało na strumieniach.

Spark → to zunifikowany silnik analityczny do obliczeń rozproszonych

Porównanie → Spark **Structured Streaming** opiera się na silniku Spark SQL

	Spark SQL	Structured Streaming
Abstrakcje danych:	DataFrame/Dataset	DataFrame/Dataset strumieniowy
	batch	micro-batch
Sięganie do źródła danych: (po utworzeniu sesji sparkowej)	spark.read.format	spark.readStream.format
Zapisywanie danych:	df.write.format	df.writeStream.format
Operacje transformacji:	select/where/groupBy	select/where/groupBy

Z punktu widzenia API → Spark wprowadza abstrakcję zbioru czy **tabeli** do której dane napływają

Na takiej wirtualnej tabeli pracuje się tak samo jak przy batch – można korzystać z nazwanych kolumn, odwoływać się do tych kolumn, wykonać filtrację czy agregację.

Spark **Structured Streaming** opiera się na mico-batch – tzn. zbiera dane w interwałach czasowych np. kilka sekund i dopiero po tym czasie zwróci wynik . Powoduje to, że opóźnienie jest wyższe niż milisekundy.

Spark

Źródła wejściowe:

Źródło pliku	odczytuje pliki zapisane w katalogu jako strumień danych. Pliki będą przetwarzane w kolejności czasu modyfikacji pliku. Jeśli latestFirst jest ustawione, kolejność zostanie odwrócona. Obsługiwane formaty plików to tekst, CSV, JSON, ORC, Parkiet. Pliki muszą być niepodzielnie umieszczone w danym katalogu, co w większości systemów plików można osiągnąć za pomocą operacji przenoszenia plików.
Źródło Kafki	Odczytuje dane z Kafki. Jest kompatybilny z brokerem Kafka w wersji 0.10.0 lub wyższej.
Źródło gniazda (do testowania)	odczytuje dane tekstowe UTF8 z połączenia gniazda. Gniazdo serwera nasłuchującego znajduje się w sterowniku. Należy zauważyć, że powinno to być używane tylko do testowania, ponieważ nie zapewnia to pełnej gwarancji odporności na awarie.
Źródło Rate (do testowania)	generuje dane z określoną liczbą wierszy na sekundę, każdy wiersz wyjściowy zawiera timestampi value.

W przetwarzaniu strumieniowym można mówić o różnym typie przetwarzania:

Typy przetwarzania strumieniowego ⇨ Stateless vs Stateful

- **stateless** — To przetwarzanie w którym każda krotka przetwarzana jest osobno – tzn. nie musimy utrzymywać stanu przetwarzania (między operacjami mikrobatch) i nie można mówić tutaj o agregacji danych
- **stateful** — W tym typie przetwarzania możliwa jest agregacja danych w określonych interwałach czasowych np. agregacja interakcji użytkownika na danej stronie. Stany odkładane w postaci CheckPoint również na wypadek awarii, korzystając przy tym z mechanizmów . Grupowanie może być np. po czasie czy kolumnie.

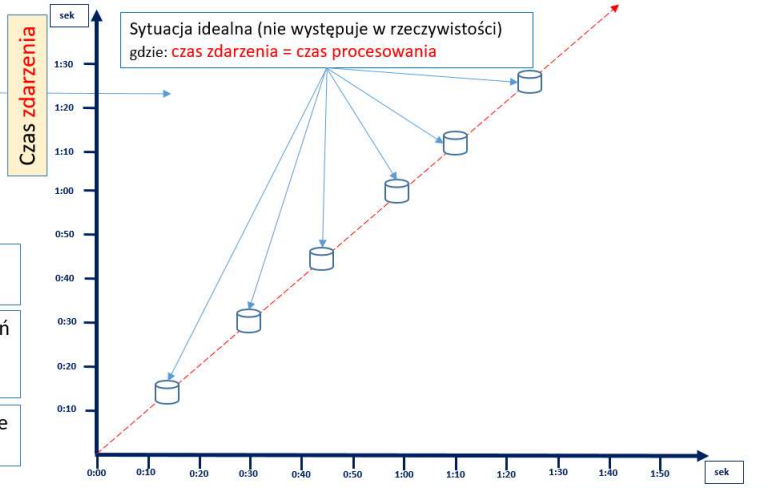
Aspekt czasu przetwarzania

- **Event time** — Czas w którym zdarzenie się odbyło, czas w którym pojawił się event.
- **Processing time** — To zdarzenie trafi do przetworzenia w innym czasie niż się pojawił (z racji opóźnień na sieci)

Obsługa czasu zdarzenia i późnych danych

Sytuacja idealna (nie występuje w rzeczywistości):

event time vs processing time



Czas zdarzenia to czas osadzony w samych danych.

Event-time jest bardzo naturalnie wyrażony w modelu

- każde zdarzenie z urządzeń to wiersz w tabeli, a event-time to wartość kolumny w wierszu.

Dzięki temu agregacje okienkowe (np. liczba zdarzeń na minutę) mogą być tylko specjalnym rodzajem grupowania i agregacji w kolumnie czasu zdarzenia

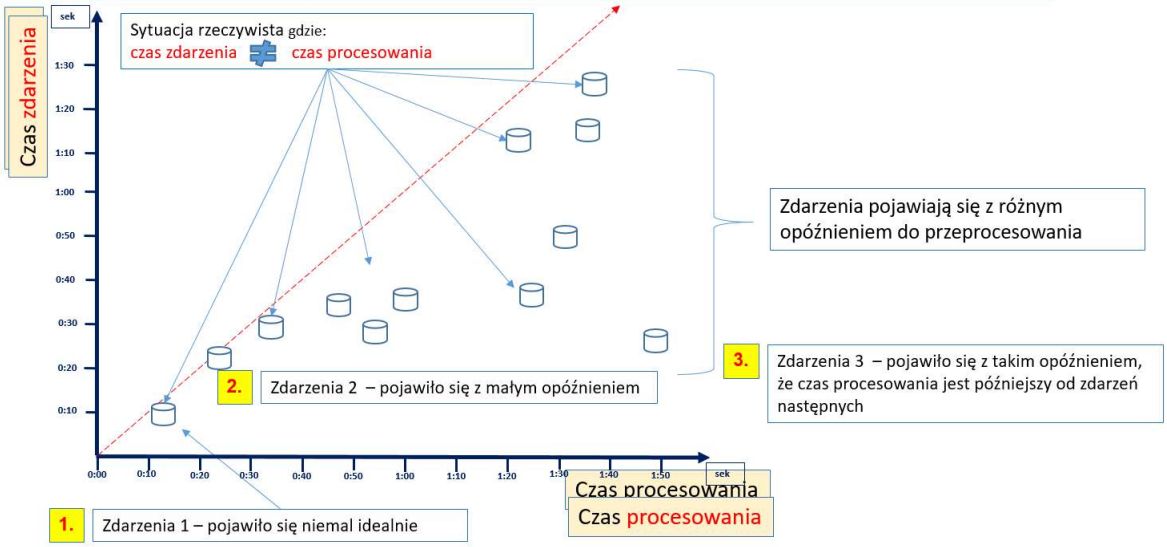
każde okno czasowe jest grupą, a każdy wiersz może należeć do wielu okien/grup.

Co więcej, model ten w naturalny sposób obsługuje dane, które nadeszły później niż oczekiwano na podstawie czasu zdarzenia. Ponieważ Spark aktualizuje tabelę wyników, ma pełną kontrolę nad aktualizowaniem starych agregatów w przypadku opóźnionych danych, a także czyszczenie starych agregatów w celu ograniczenia rozmiaru danych stanu pośredniego.

Sytuacja rzeczywista.

event time vs processing time

Zwróćmy uwagę, że istotny jest czas pojawienia się zdarzenia do przeprocesowania i kolejność pojawienia się zdarzenia

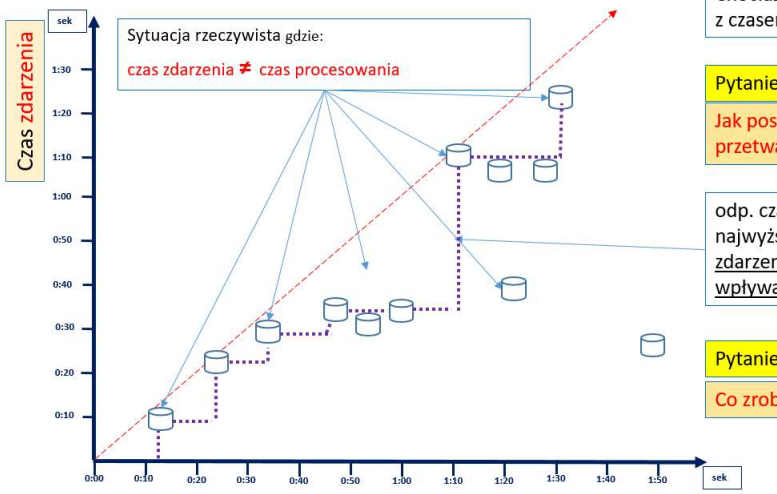


UWAGA:

Jeżeli przetwarzali byśmy zgodnie z czasem procesowania a nie zgodnie z czasem zdarzeń to – była by to zupełnie inna kolejność sekwencji.

Sytuacja rzeczywista.

Często zależy nam na czasie przetwarzania zgodnie z kolejnością zdarzeń a nie czasu procesowania.
Powoduje to, że będziemy mieli zdarzenia ułożone w kolejności w których faktycznie te zdarzenia się wydarzyły.



Chociaż Spark pozwala również na przetwarzanie zgodnie z czasem procesowania.

Pytanie:

Jak posuwa się czas procesowania zdarzeń (dla przetwarzania zgodnie z kolejnością zdarzeń)?

odp. czas posuwa się razem ze zdarzeniem, które ma najwyższy timestamp tzn. zgodnie z najświeższym zdarzeniem natomiast zdarzenia, które się spóźniły nie wpływają na to, że zdarzenie i czas się cofa.

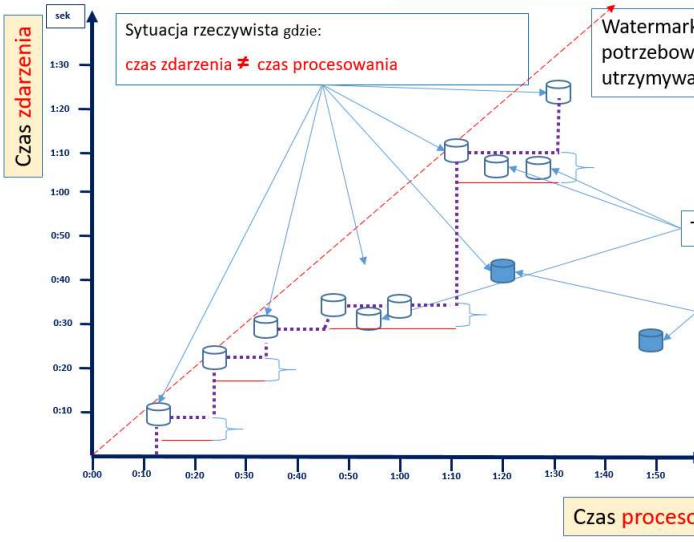
Pytanie:

Co zrobić ze zdarzeniami, które się opóźniły?

Operacje na oknach czasowych

Spark jako metodę radzenia sobie z opóźnionymi zdarzeniami oferuje watermark.

To nic innego jak metoda definiująca jak bardzo może być opóźnione zdarzenie by można było je traktować jako zdarzenia poprawne – czyli ile czasu może być opóźnione zdarzenia, by były włączone do przetworzenia (np. jakieś agregacji).



Watermark – musi uwzględniać również aspekt jak dużo pamięci będziemy potrzebowali przy agregacji opóźnionych zdarzeń. (Stan musi być utrzymywany przy agregowaniu np. w ognie 10 sekundowym).

Te trzy zdarzenia wejdą do okna czasowego przetwarzania

Te dwa zdarzenia nie wejdą do okna czasowego przetwarzania

Operacje na oknach czasowych

Operacje na oknach w czasie zdarzenia

https://spark.apache.org/docs/latest/streaming-programming-guide.html

Operacje na oknach czasowych:

- okno przesuwne (sliding window)
- spadające okno (tumbling window)

A.

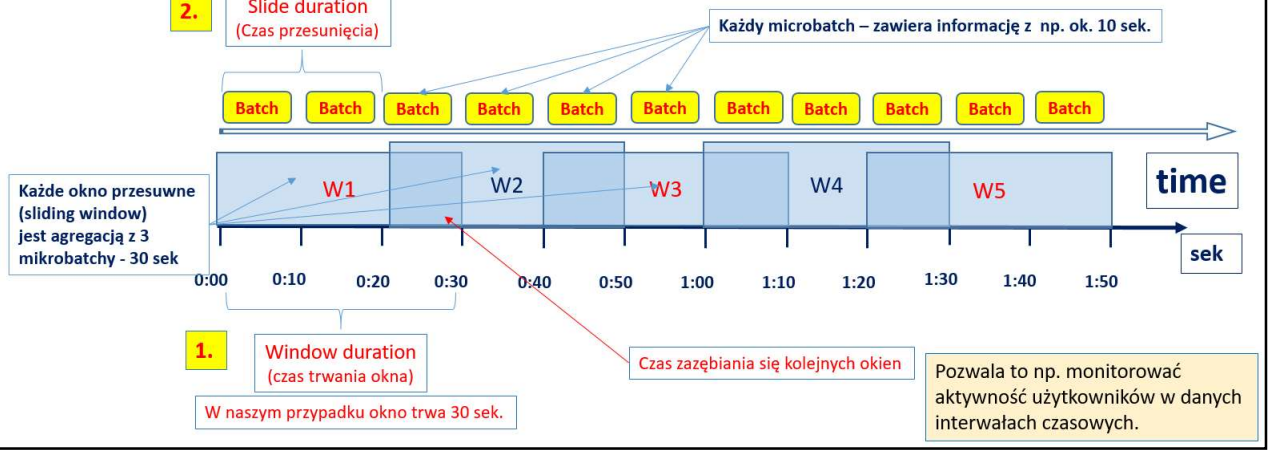
- okna sesyjne (session window)

Okno przesuwne (sliding window) → To sytuacja w którym **Window duration > Slide duration**

- definiujemy jak długie to okno ma być. — Np.. Okno 30 sekund → określa tzw. Window duration
- definiujemy jakie jest przesunięcie okna . — Np. Okno 20 sekund → określa tzw. Slide duration

W naszym przypadku okno przesuwa się o 20 sek.

2. Slide duration (Czas przesunięcia)

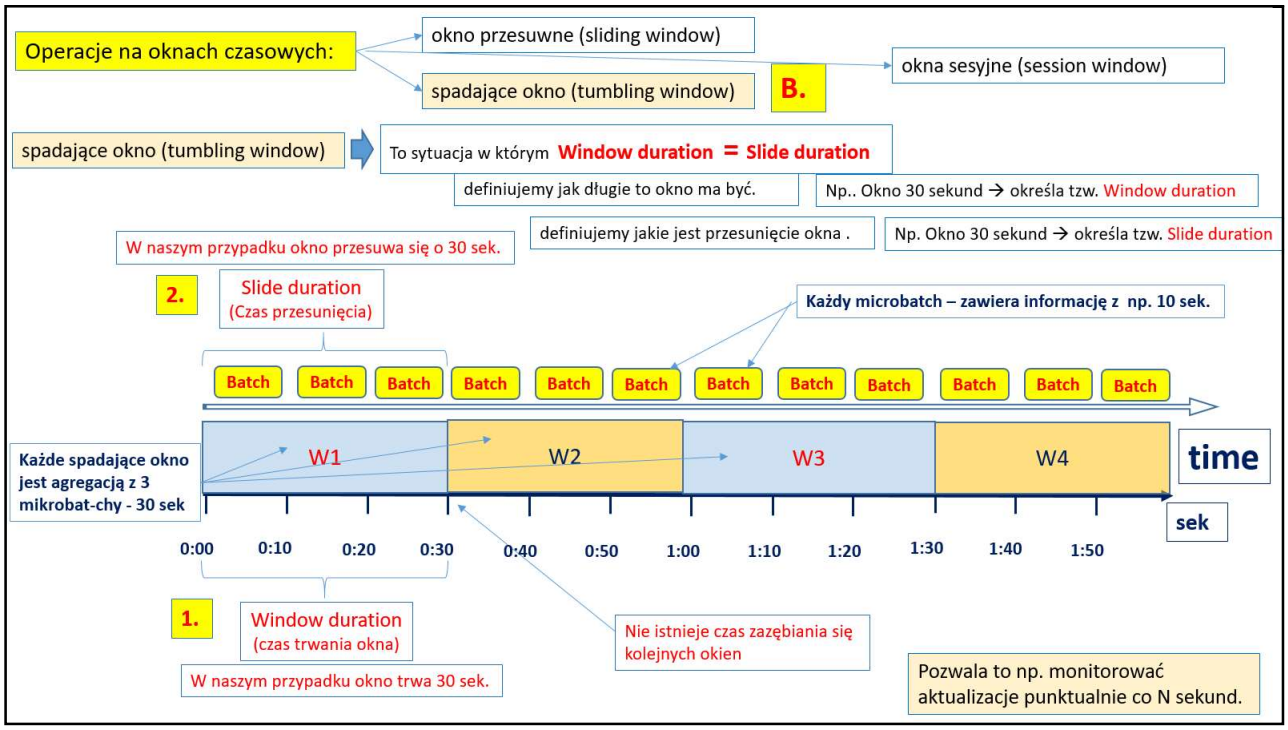


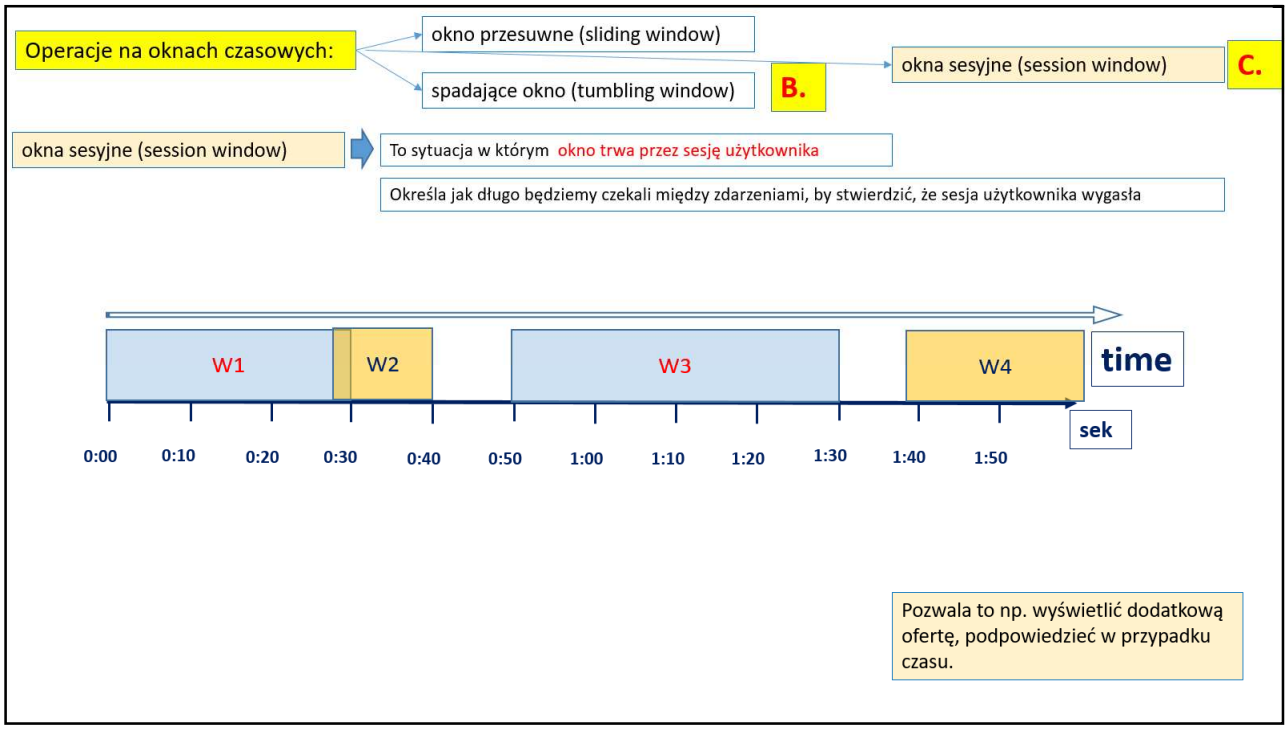
1. Window duration (czas trwania okna)

W naszym przypadku okno trwa 30 sek.

Czas zazębiania się kolejnych okien

Pozwala to np. monitorować aktywność użytkowników w danych interwałach czasowych.


Operacje na oknach czasowych:
okno przesuwne (sliding window)
spadające okno (tumbling window)
B.
okna sesyjne (session window)
spadające okno (tumbling window)
To sytuacja w którym Window duration = Slide duration
definiujemy jak długie to okno ma być.
Np.. Okno 30 sekund → określa tzw. Window duration
definiujemy jakie jest przesunięcie okna .
Np. Okno 30 sekund → określa tzw. Slide duration
W naszym przypadku okno przesuwa się o 30 sek.
2.
Slide duration
(Czas przesunięcia)
Każdy microbatch – zawiera informację z np. 10 sek.
Batch
W1
W2
W3
W4
time
sek
Każde spadające okno jest agregacją z 3 mikrobat-chy - 30 sek
0:00
0:10
0:20
0:30
0:40
0:50
1:00
1:10
1:20
1:30
1:40
1:50
1.
Window duration
(czas trwania okna)
W naszym przypadku okno trwa 30 sek.
Nie istnieje czas zazębiania się kolejnych okien
Pozwala to np. monitorować aktualizacje punktualnie co N sekund.


Operacje na oknach czasowych:
okno przesuwne (sliding window)
spadające okno (tumbling window)
B.
okna sesyjne (session window)
C.
okna sesyjne (session window)
To sytuacja w którym okno trwa przez sesję użytkownika
Określa jak długo będziemy czekali między zdarzeniami, by stwierdzić, że sesja użytkownika wygasła
W1
W2
W3
W4
time
sek
0:00
0:10
0:20
0:30
0:40
0:50
1:00
1:10
1:20
1:30
1:40
1:50
Pozwala to np. wyświetlić dodatkową ofertę, podpowiedzieć w przypadku czasu.

Operacje na oknach czasowych

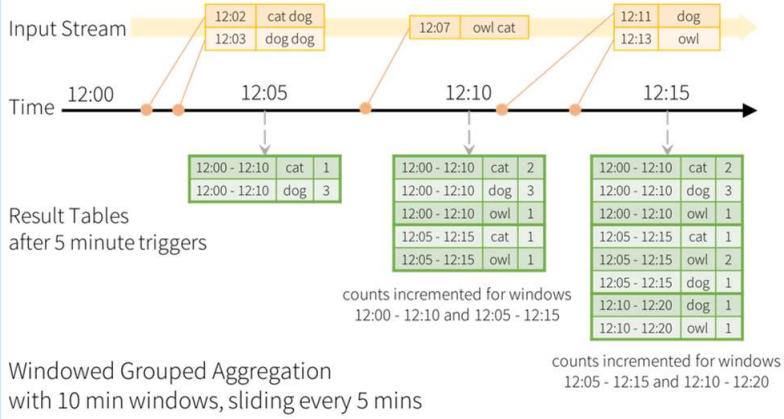
Agregacje w przesuwanym oknie czasu zdarzenia są proste w przypadku Strumieniowania i są bardzo podobne do agregacji zgrupowanych.

W agregacji zgrupowanej wartości zagregowane (np. krotności) są zachowywane dla każdej unikatowej wartości w kolumnie grupowania określonej przez użytkownika. W przypadku agregacji opartych na oknach, wartości zagregowane są utrzymywane dla każdego okna, w którym przypada czas zdarzenia wiersza.

chcemy zliczać słowa w 10-minutowych oknach, aktualizując je co 5 minut.

Oznacza to, że liczba słów w słowach otrzymanych między oknami 10-minutowymi 12:00 - 12:10, 12:05 - 12:15, 12:10 - 12:20 itd.

Należy pamiętać, że 12:00 - 12:10 oznacza dane, które przybył po 12:00, ale przed 12:10. Zastanówmy się teraz nad słowem otrzymanym o 12:07. To słowo powinno zwiększać zliczenia odpowiadające dwóm oknom 12:00 - 12:10 i 12:05 - 12:15. Tak więc zliczenia będą indeksowane zarówno przez klucz grupujący (tj. słowo) jak i okno (można obliczyć na podstawie czasu zdarzenia).



https://spark.apache.org/docs/3.1.2/structured-streaming-programming-guide.html#window-operations-on-event-time

Przykłady:

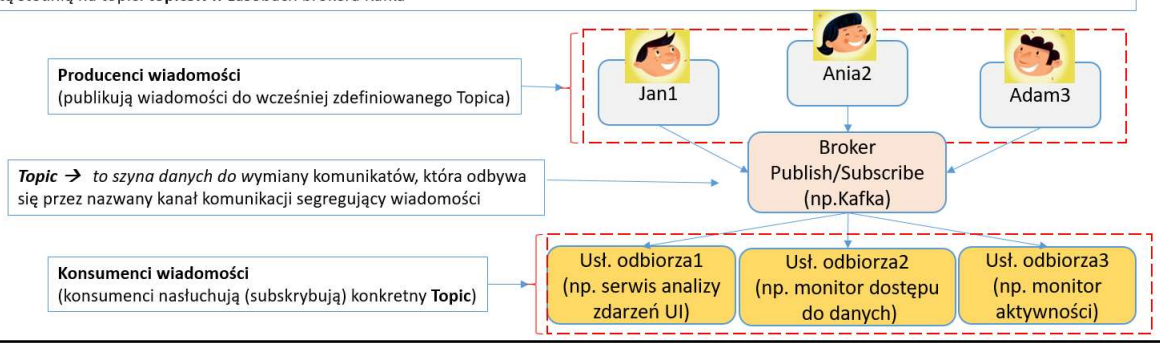
Wysyłanie wiadomości do brokera Kafka:

Wysyłanie wiadomości do brokera Kafka:

Zadana:

Wykorzystując pySpark opracować:

1. - skrypt python, który prześle do brokera Kafka (co sekundę) na topic: **topicBD** informację o ocenach pięciu studentów – Oceny całkowite z zakresu 2-5
2. - skrypt, który odbierze do brokera Kafka z topic: **topicBD** informację o zdarzeniach danych użytkowników
3. - skrypt, który odbierze do brokera Kafka z topic: **topicBD** i zaprezentuje działanie sliding window
4. - skrypt, który odbierze do brokera Kafka z topic: **topicBD** i zaprezentuje działanie tumbling window
5. - skrypt, który odbierze do brokera Kafka z topic: **topicBD** i zaprezentuje działanie watermark
6. - skrypt, który odbierze do brokera Kafka z topic: **topicBD** wykona agregację wyliczającą średnią ocenę uzyskaną przez danego studenta oraz zapisze tą średnią na topic: **topicSR** w zasobach brokera Kafka



Przykłady:

Na podstawie prezentacji w ramach grupy społecznościowej Meetup – Big Data Community

Spotkanie Meetup: Spark Structured Streaming

Prezentujący: **Patryk Pilarski**

Kod w języku python –do przesyłania wiadomości przez brokera Kafka

W PyCharm → strumień generowany jest przez Kafkę, do której generowane są wiadomości przy pomocy kodu napisanego w python.

```
import json
import random
import sys
from datetime import datetime, timedelta
from time import sleep

from kafka import KafkaProducer


if __name__ == "__main__":
    server = sys.argv[1] if len(sys.argv) == 2 else "localhost:9092"

    producer = KafkaProducer(
        bootstrap_servers=[server],
        value_serializer=lambda x: json.dumps(x).encode("utf-8"),
        api_version=(2, 7, 0),
    )

    try:
        while True:
            message = {
                "time": str(
                    datetime.now() + timedelta(seconds=random.randint(-15, 0))
                ),
                "id": random.choice(["Jan1", "Ania2", "Adam3", "Alan4", "Zofia5"]),
                "value": random.randint(2, 5),
            }
            producer.send("topicBD", value=message)
            sleep(1)
    except KeyboardInterrupt:
        producer.close()
```

Importy niezbędnych bibliotek → **import sys** potrzebny, by można było aplikację uruchomić z argumentami podawanymi w komendzie uruchomieniowej do klastra Kafka (w przeciwnym razie stosowany jest **localhost**

Producer – generuje JSON kodowanym UTF-8 przez niekończącą się pętlę while:

json.dumps(x) - konwertuje obiekty Pythona (tutaj x) na ciąg Sting. Istnieje również metoda **dump()** → konwertuje obiekty pythona na odpowiednie obiekty json.

Wiadomości składające się:

- ze znacznika timestamp → potrzebny do zasymulowania czasu zdarzenie (dla event) z losowym opóźnieniem z zakresu 1- 15 sekund.
- pobranie wartości randomowych stanowiący id i symulujący dany rekord.
- metoda send() – pozwala wysłać na topic zdefiniowaną wartość wiadomości
- Wysyłanie dobywa się co jedną sekundę.

Łączenie się Apache Spark z brokerem Kafka:

Kod w języku python –do przesyłania wiadomości przez broker Kafka

W **PyCharm** → strumień generowany jest przez Kafkę, do której generowane są wiadomości przy pomocy kodu napisanego w **python**.

```
# spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.1.1 1_app.py host:9092
import sys

from pyspark.sql import SparkSession


if __name__ == "__main__":
    server = sys.argv[1] if len(sys.argv) == 2 else "localhost:9092"

    spark = SparkSession.builder.appName("streamKafka").getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")

    # topic subscription
    raw = (
        spark.readStream.format("kafka")
        .option("kafka.bootstrap.servers", server)
        .option("subscribe", "topicBD")
        .load()
    )

    # defining output
    query = raw.writeStream.outputMode("append").format("console").start()
    query.awaitTermination(60)
    query.stop()
```

W komentarzu przykład komendy do uruchomienia aplikacji

Import biblioteki sesji sparkowej

Przygotowanie sesji Spark (podobnie jak na batch-ach)

Ograniczenie zgłaszania komunikatów tylko do błędów

Definicja subskrypcji do Apache Kafka. Do czytania danych sięgamy po sesję sparkową, następnie metoda **readStream** i wskazujemy nasz format – ustawiony na Kafka.

Kafka – wymaga kilku opcji – w których musimy wskazać ***kafka.bootstrap.servers*** i wskazać namiary na jeden z węzłów **node** w naszym klastrze Kafka lub zestaw węzłów **node** w klastrze Kafka.

Wskazujemy który **topic** chcemy zasubskrybować (może to być lista topic) można stosować też **regexp**.

load() – pobiera napływające z Kafki z wiadomości tworząc na wyjściu w zmiennej **row** strumieniowy **df**

nasz **df** – będziemy zwracali do konsoli (ale można zwracać do bazy lub innego topic w Kafce

outputMode: ma trzy typy:

- **Append** – zwraca tylko nowe wiersze (np. bez agregacji)
- **Update**– gdy chcemy zwracać zmienione agregaty od ostatniego output-u
- **Complete**– zwraca cały stan naszego strumienia w danym momencie.

Po 60 sekundach przerywane jest pobieranie strumienia

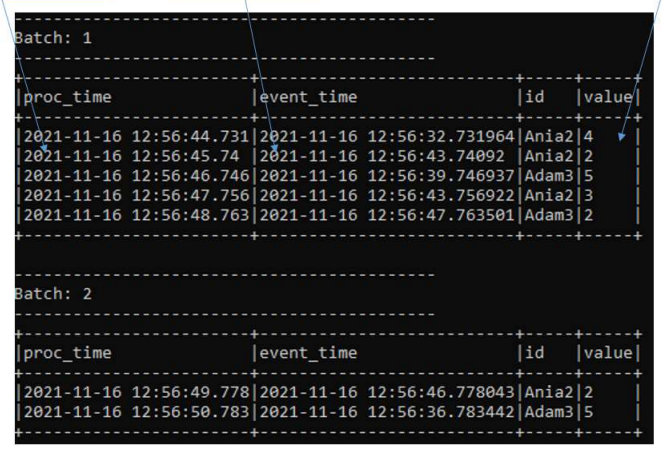
Przykłady:

Wiadomości z Kafki:

Wartości które są zdekodowane:

Czas procesowania

Czas zdarzenia



```
-------------------------------------------
Batch: 1
-------------------------------------------
+-----------------------+-------------------------+-----+-----+
|proc_time              |event_time               |id   |value|
+-----------------------+-------------------------+-----+-----+
|2021-11-16 12:56:44.731|2021-11-16 12:56:32.731964|Ania2|4    |
|2021-11-16 12:56:45.74 |2021-11-16 12:56:43.74092 |Ania2|2    |
|2021-11-16 12:56:46.746|2021-11-16 12:56:39.746937|Adam3|5    |
|2021-11-16 12:56:47.756|2021-11-16 12:56:43.756922|Ania2|3    |
|2021-11-16 12:56:48.763|2021-11-16 12:56:47.763501|Adam3|2    |
+-----------------------+-------------------------+-----+-----+

-------------------------------------------
Batch: 2
-------------------------------------------
+-----------------------+-------------------------+-----+-----+
|proc_time              |event_time               |id   |value|
+-----------------------+-------------------------+-----+-----+
|2021-11-16 12:56:49.778|2021-11-16 12:56:46.778043|Ania2|2    |
|2021-11-16 12:56:50.783|2021-11-16 12:56:36.783442|Adam3|5    |
+-----------------------+-------------------------+-----+-----+
```